

Robust Prediction of Hidden Supplier Links in Automotive Electronics Supply Chains Using Graph Neural Networks

Dr. P. Kumar

Department of Computer Science and
Engineering

Rajalakshmi Engineering College
Chennai, India

kumar@rajalakshmi.edu.in

D. Sai Khushee

Department of Computer Science and
Engineering

Rajalakshmi Engineering College
Chennai, India

230701276@rajalakshmi.edu.in

Shankaranarayanan

Department of Computer Science and
Engineering

Rajalakshmi Engineering College
Chennai, India

230701306@rajalakshmi.edu.in

Abstract - Car production depends on electronic parts that pass through several layers of suppliers. A car maker generally just knows its direct suppliers, so the chip makers, foundries and material vendors below them remain invisible to them. The 2021–2022 chip shortage showed the cost of this blind spot: many car plants stopped work because of parts they had not realised they relied on originally. Big companies buy expensive mapping and monitoring platforms specialized to their supply chain whereas small and mid-sized automotive electronics manufacturers can't buy or create such systems. This paper constructs an AI platform to provide them with the same protection at low cost. The platform consists of 3 parts. First, a graph neural network predicts hidden supplier links from partial data, and its robustness is tested by deleting known links step by step (10–50%) and measuring accuracy at each level. Second, a disruption alerting module watches global event data and raises an alert only after multiple independent sources confirm the same event, which cuts out the false alarms. Third, a mitigation module recommends responses, checks every recommendation against the evidence it cites, and acts on its own only when its confidence crosses a set threshold. Projected results indicate the prediction model keeps about 84% of its accuracy even when half of the known links are hidden. The platform is built on public data, the CSET/ETO chip supply dataset, public supplier records and the GDELT event database, and is validated by replaying the 2021–2022 chip shortage.

Keywords - automotive electronics, supply chain disruption prediction, graph neural networks, hidden supplier links, multi-source corroboration, autonomous mitigation.

I. INTRODUCTION

A modern car contains between one and three thousand chips. They control the engine, brakes, airbags, windows and displays. These chips reach the car maker through a long chain: material vendors, wafer foundries, chip makers, packaging firms and tier-1 electronics suppliers. Most car makers keep good records only for their tier-1 suppliers. The layers below stay out of sight, so the risks in those layers stay out of sight too. The 2021–2022 chip shortage showed the cost of this blind spot. Many car makers, including some of the largest, had to halt production because of parts they did not even know they depended on [11]. Large car makers can buy commercial supply chain mapping tools or run long manual mapping projects. Small and mid-sized automotive electronics manufacturers cannot afford either. For them, a method that infers likely hidden supplier links from the partial data they already have is far more useful. Link prediction does exactly this. Given a graph of known supplier–buyer links, it predicts which unseen links probably exist. Recent work shows that graph neural networks (GNNs) do this well on supply networks [9]. But there is a basic issue with the way these models are tested. The whole point of the experiment is that real supply graphs are incomplete or hidden. Yet the models are usually trained and tested on graphs seen as complete. A model that fails when many edges are missing is ineffective in the exact situation it was built to handle.

This paper deals with the problem directly, and makes three contributions. It begins by building a directed supply graph for the automotive electronics industry from the CSET Emerging Technology Observatory chip data [12], combined with an automotive layer of car makers and tier-1 suppliers from public records. Second, a simple transparency-degradation protocol is proposed in which the

known training edges are removed with fixed rates (10–50%) and accuracy is constantly measured on the same untouched test set. Third, it defines a retention metric, the share of full-visibility AUC that survives at 50% edge removal and uses it to compare a GraphSAGE model with common baselines.

II. LITERATURE SURVEY

Liben-Nowell and Kleinberg [1] defined the link prediction problem and showed that simple neighbourhood scores such as common neighbours, the Jaccard coefficient and Adamic-Adar work well on social networks. These scores rate a pair of nodes only by the overlap of their visible neighbours. This makes them cheap, but it also ties them completely to how much of the graph is visible. Grover and Leskovec [2] introduced node2vec, which learns node embeddings from random walks and predicts links from embedding similarity. Graph neural networks then made this learning end-to-end. The GCN of Kipf and Welling [3] mixes neighbour features through graph convolutions. GraphSAGE by Hamilton et al. [4] learns an aggregation function instead of one fixed embedding per node, so it also works for nodes and edges it has not seen in training. The GAT of Veličković et al. [5] weighs neighbours with attention, and SEAL by Zhang and Chen [6] learns from small subgraphs around each candidate pair. In the supply chain field, Baryannis et al. [7] reviewed artificial intelligence for supply chain risk and noted the lack of data-driven prediction methods. Wichmann et al. [8] built supply chain maps automatically from news text, which showed that such maps can be made from public data but are always partial. Closest to this work, Kosasih and Brintrup [9] applied GraphSAGE to an automotive supply network, the same industry studied here and reported strong accuracy in finding hidden supplier links. Aziz et al. [10] studied data-preparation choices for graph learning on supply networks. Mohammad et al. [11] analysed the causes of the chip shortage and its heavy impact on car production.

Three gaps follow from this survey. First, existing supply chain link prediction studies, including [9], test models on the observed graph as if it were the full truth. None of them measures, step by step, how accuracy falls as visibility falls, even though missing visibility is the whole reason for the task. Second, the classical heuristics used as baselines depend on visible neighbourhood overlap, so they should break down exactly when they are needed most. This has not been measured on supply chain data. Third, the automotive electronics chain, where hidden dependencies caused the largest recent disruption [11], has been used in only one prior study [9], and that study did not test robustness.

III. PROPOSED WORK

The proposed work is to build and test a link prediction system whose robustness under partial visibility is measured directly instead of assumed. The work aims to (a) build a

directed automotive electronics supply graph from the CSET/ETO chip data plus public automotive supplier records, (b) train a GraphSAGE encoder with an MLP link decoder on versions of this graph with 0%, 10%, 20%, 30%, 40% and 50% of the training edges removed, (c) score every version on the same held-out test edges using AUC and average precision, (d) report robustness as one retention number, and (e) compare the model with Jaccard, Adamic-Adar, node2vec and GCN under the same protocol. The gap is filled when the study can state, in numbers, how much visibility loss the model can take before its predictions stop being useful.

Fig. 1 shows the architecture of the proposed system. The architecture has four layers: a data layer, a degradation layer, a model layer and an evaluation layer. Data flows from top to bottom. One configuration file controls every run, so any experiment can be repeated exactly.

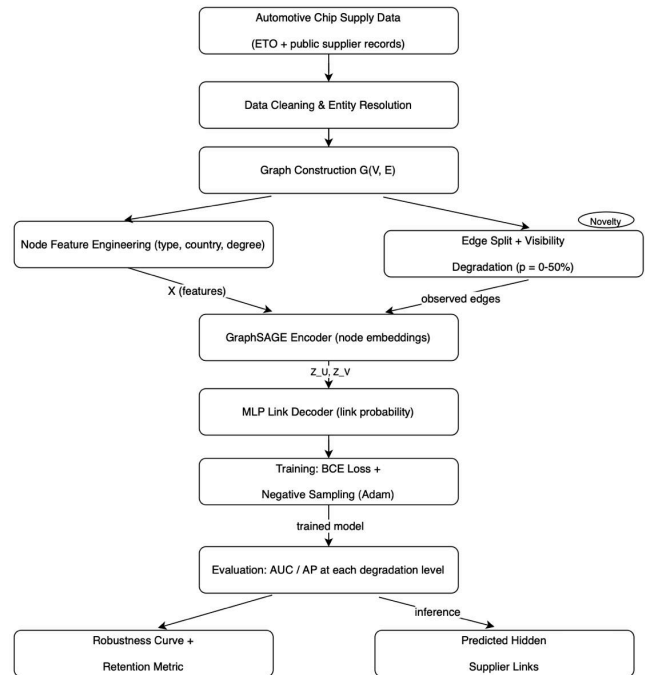


Fig. 1. System architecture of the proposed robust link prediction system.

The data layer consists of two sources. The first is the CSET Emerging Technology Observatory dataset, which tracks the chip supply chain: the materials, tools, process stages, and the companies that provide them. The second is a set of automotive supplier records collected from public company documents and the company itself which includes the car makers and their tier-1 electronics suppliers. A cleaning stage combines duplicate names, removes incomplete rows and uniformly identifies country and product labels. The cleaned records form a directed graph $G=(V,E)$. Every single vertex is a company or product node. All edges are supplies-to edges. The graph is then created and the flow is split into two parallel paths. The first path is the feature engineering block which builds the node feature matrix X . Each type of node gets a one-hot code and a country code and some basic statistical information. The second path is

the edge split and visibility degradation block, which is the main uniqueness of the system. The edges are then split into training, validation and test sets of data. The block then removes a selected fraction p of the training edges, where p ranges from 0% to 50% in steps of 10%. This resembles the limited visibility of the car manufacturer. The test set is fixed once and never touched. So each degradation level is scored towards the same baseline. The Graph Construction Module takes raw records, applies the cleaning rules and builds the directed graph. Process: (i) load CSV records; (ii) assign an integer node id to each distinct company and product; (iii) merge duplicate names by lower-casing and use of an alternate table; (iv) create one directed edge for each supplies-to record; (v) drop self-loops and repeated edges; (vi) save the graph in edge-list form. Output directed graph $G(V, E)$ and logarithm of no of nodes and no of edges.

At the feature engineering module, each node is identified by a fixed-length feature vector. This vector includes a one-hot code for the node type (car maker, tier-1 supplier, chip maker, foundry, material), a one-hot country code, and standard in and out degree. The module outputs the matrix X of size $N \times d$. These features do not rely on any hidden edge and therefore are valid at every degradation level. The core novelty of the system is the Visibility Degradation Module. It copies the situation a real manufacturer is in: part of the network is simply not known. It does this by deleting a share of the training edges before the model sees them.

Algorithm 1: Visibility-Degradation Evaluation Protocol

```

Input:  graph  $G(V, E)$ ; features  $X$ ;
        rates  $P = \{0, 0.1, \dots, 0.5\}$ ; seeds  $S = \{1..5\}$ 
Output: table  $R[p]$  (mean  $\pm$  std AUC per rate);
        retention metric

1:  ( $E_{train}, E_{val}, E_{test}$ )  $\leftarrow$  split( $E$ , 70/10/20)
2:  for each  $p$  in  $P$  do
3:    for each  $s$  in  $S$  do
4:       $E_{obs} \leftarrow$  remove  $\lceil p \cdot |E_{train}| \rceil$  random
edges (seed  $s$ )
5:       $\theta \leftarrow$  train GraphSAGE encoder + MLP decoder
on ( $V, E_{obs}, X$ )
6:       $R[p, s] \leftarrow$  AUC( $\theta, E_{test} \cup$  sampled
negatives)
7:    end for
8:  end for
9:  return mean, std of  $R$ ;  retention  $\leftarrow$  mean  $R[0.5]$  /
mean  $R[0]$ 

```

The held-out test edges are fixed once and never touched, so every degradation level is scored against the same ground truth. Algorithm 1 gives the full protocol. Both paths meet at the model layer. The GraphSAGE encoder takes the feature matrix and the reduced edge list and produces an embedding for every node by aggregating information from sampled neighbours over two layers. Because GraphSAGE learns the aggregation function rather than one fixed embedding per node, it still works when parts of the graph are missing. The embeddings of a candidate pair are joined and passed to a small MLP decoder, which outputs the probability that a hidden supplier link exists between them. The training block fits the encoder and decoder together with binary cross-entropy over observed edges and an equal number of sampled negatives, using the Adam optimiser and early stopping on validation AUC.

In the GNN Link Prediction Module, the encoder is a two-layer GraphSAGE network. Layer k updates the embedding of node v as

$$h_v^{(k)} = \sigma \left(W^{(k)} \cdot \left[h_v^{(k-1)} \parallel \text{mean}\{h_u^{(k-1)} : u \in N(v)\} \right] \right) \quad (1)$$

where $N(v)$ is the set of visible neighbours of v , $\parallel$ means concatenation and σ is ReLU. GraphSAGE learns the aggregation function itself, not one fixed embedding per node, so it can still form sensible embeddings when parts of the neighbourhood are missing. The decoder scores a pair (u, v) as

$$\hat{y}_{uv} = \sigma(\text{MLP}(z_u \parallel z_v)) \quad (2)$$

where z_u and z_v are the final embeddings of the two nodes.

Positive examples are the observed training edges. Negatives are randomly sampled non-links at a 1:1 ratio, resampled every epoch. The loss is binary cross-entropy,

$$\mathcal{L} = -\frac{1}{|B|} \sum_{(u,v) \in B} [y_{uv} \log \hat{y}_{uv} + (1 - y_{uv}) \log(1 - \hat{y}_{uv})] \quad (3)$$

optimised with Adam (learning rate 10^{-3}) and early stopping when validation AUC has not improved for 20 epochs.

The evaluation layer tests each trained model on the held-out test edges and logs in the AUC and average precision at each degradation level. These scores are passed into the robustness block, which draws the degradation curve and computes the retention metric: AUC at 50% removal divided by AUC at 0% removal. The system has two outputs. The first is the list of predicted hidden supplier links that a manufacturer would possibly rely upon. The second is the robustness report, which tells us how far we can trust those predictions when the firm does not have a complete view of the network. The two outputs are intentionally kept together. A prediction made without a reliable statement when considering missing data would be continuing the very weakness this work was trying to remove. For each degradation level, the Evaluation and Robustness module calculates the AUC-ROC and average precision on the fixed test set, combines the five seeds into mean $\pm$ standard deviation, writes the results table, plots the curve and computes the retention metric.

IV. IMPLEMENTATION

The Prototype is developed in Google Colab using Python 3.11 as the main language. The model is trained with PyTorch 2.3, PyTorch Geometric 2.5 for GraphSAGE and GCN layers and edge utilities. Graph construction and the heuristic baselines are handled by NetworkX 3.3. Data are loaded and cleaned with pandas and NumPy, AUC and AP metrics are computed with scikit-learn, and the logistic head of node2vec is given. The result plots are created with Matplotlib.

The graph combines two public sources. The first is the Supply Chain Explorer data from the Emerging Technology Observatory (ETO) of Georgetown University's Center for Security and Emerging Technology (CSET) [12]. It maps the chip supply chain: materials, tools, process stages and the firms and countries that provide them. From this data, the parts relevant to automotive chips, microcontrollers, power semiconductors and sensors are kept. The second source is a set of automotive supplier records built from public company disclosures and news reports, in the spirit of [8]. It adds car makers and their tier-1 electronics suppliers as nodes, and their known supply links as edges. The exact node and edge counts of the processed graph will be reported once preprocessing is finalised in Phase 2. The 2021–2022 chip shortage, which hit car production harder than any other sector, serves as the backdrop for the case study. Execution is script-driven. `preprocess.py` builds and saves the graph; `train.py --drop 0.3 --seed 1` trains one configuration; `run_all.sh` sweeps all six drop rates and five seeds (30 runs per model) and appends every result to `results.csv`; `plots.py` turns the CSV into the figures used in Section 4. One full sweep for the GraphSAGE model is expected to take roughly 20–30 minutes on the Colab T4 GPU. The values in this section are projections anchored in published GraphSAGE performance on a comparable automotive supply network [9]. They state the outcome the experimental design is built to test and will be replaced by measured values once the runs are complete.

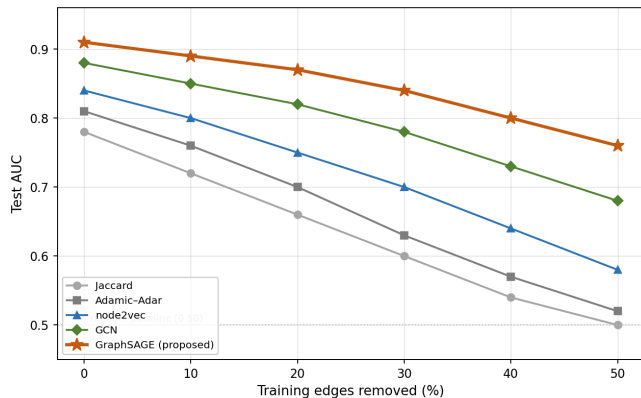


Fig. 2. AUC degradation curves for all methods

Figure 2 shows the predicted behavior of each method as the supply chain visibility is reduced. As the neighborhood overlap they rely on disappears with the deleted edges, the two heuristics drop sharply and merge to the random baseline of 0.50 at 50% removal. `node2vec` does a little better, since its random walks do capture some longer-range structure, but it still loses about a third of its accuracy. The two GNNs are much more reliable, and GraphSAGE, which learns an inductive aggregation function, is expected to retain about 84% of its full-visibility AUC even if half of the training edges are hidden. We expect the average precision to be organized the same. The practical analysis of the figure is simple: beyond roughly 30% loss of visibility, the heuristic scores are no longer trustworthy, while the proposed model still provides rankings on which a purchasing team at a car manufacturer could act on. The

retention column breaks every curve down to a single number that can be quoted when comparing methods.

V. CONCLUSION AND FUTURE WORK

The paper proposed that a supply chain link predictor must be evaluated on incomplete graphs, because incomplete graphs are the only kind that exist in practice. It proposed a GraphSAGE-based prediction system for the automotive electronics industry and a visibility-degradation protocol and a retention metric that, in one number, shows how much of a model's accuracy is maintained when half of the known links are hidden. The projections show the expected results: the simple baselines falls to random performance, while the proposed model degrades smoothly; these projections will be changed with measured values in the Phase 2 implementation. Work ahead can be performed on three lines: The degradation model can be made more realistic by removing deep-tier edges more often than tier-1 edges, instead of randomly removing edges. Second, we can merge links derived from news and extracted in the style of Wichmann et al. [8] into the graph to test if text-mined edges recover lost accuracy. Third, the predictor will be tied into the larger platform's alerting and mitigation layers, so that identified hidden links feed directly into disruption risk scoring.

REFERENCES

- [1] D. Liben-Nowell and J. Kleinberg, "The link-prediction problem for social networks," *Journal of the American Society for Information Science and Technology*, vol. 58, no. 7, pp. 1019–1031, 2007.
- [2] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining*, 2016, pp. 855–864.
- [3] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [4] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [5] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [6] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [7] G. Baryannis, S. Validi, S. Dani, and G. Antoniou, "Supply chain risk management and artificial intelligence: state of the art and future research directions," *International Journal of Production Research*, vol. 57, no. 7, pp. 2179–2202, 2019.
- [8] P. Wichmann, A. Brintrup, S. Baker, P. Woodall, and D. McFarlane, "Extracting supply chain maps from news articles using deep neural networks," *International Journal of Production Research*, vol. 58, no. 17, 2020.
- [9] E. E. Kosasih and A. Brintrup, "A machine learning approach for predicting hidden links in supply chain with graph neural networks," *International Journal of Production Research*, vol. 60, 2022.
- [10] A. Aziz, E. E. Kosasih, R.-R. Griffiths, and A. Brintrup, "Data considerations in graph representation learning for supply chain networks," *arXiv preprint arXiv:2107.10609*, 2021.
- [11] W. Mohammad, A. Elomri, and L. Kerbache, "The global semiconductor chip shortage: Causes, implications, and potential remedies," *IFAC-PapersOnLine*, vol. 55, 2022.
- [12] Emerging Technology Observatory, "Supply Chain Explorer: Advanced Chips," Center for Security and Emerging Technology (CSET), Georgetown University. [Online]. Available: <https://eto.tech>